DOCUMENT DE CADRAGE · MODULE CLOUD AVANCÉ AVEC APPRENTISSAGE AUTOMATIQUE

Analyse automatique de la satisfaction client par agrégation hebdomadaire d'avis textuels

Cas d'étude : NordicHome, e-commerce de mobilier (entreprise fictive). Architecture AWS serverless, région eu-west-1. Infrastructure : AWS SAM (`template.yaml`).

1. Analyse du contexte

Une PME du commerce en ligne reçoit des avis en continu : formulaires après-achat, marketplaces, tickets de support. Quelques dizaines par semaine, quelques centaines par mois. Volume trop élevé pour être lu intégralement, trop faible pour justifier un data scientist ou une plateforme d'analyse spécialisée. Les avis finissent dans un tableur. On les consulte quand un client se plaint fort, ou au bilan trimestriel. Un problème de transporteur qui s'installe sur trois semaines peut passer inaperçu jusqu'à se lire dans le chiffre d'affaires.

Constat 1 — un avis isolé a peu de valeur. Savoir qu'un client est mécontent permet au mieux de traiter son cas. Savoir que 46 % des avis de la semaine sont négatifs et que le motif dominant est le retard de livraison, c'est une information actionnable : appeler le transporteur, renforcer une équipe, ajuster les délais.

Constat 2 — le coût d'entrée dans l'analyse de texte a chuté. Un service managé comme Amazon Comprehend fournit sentiment et phrases clés par appel d'API, sans modèle à entraîner, sans corpus annoté, sans GPU. Ce qui était un projet de plusieurs mois est devenu une brique d'infrastructure.

2. Problématique

> *Comment une PME sans data scientist ni infrastructure dédiée peut-elle savoir, chaque semaine, si ses clients ont été satisfaits — et pourquoi ?*

Trois exigences structurent le projet. Automatiquement : aucune intervention entre l'arrivée d'un avis et la synthèse. Sur une période : le livrable n'est pas l'avis unitaire, c'est « comment s'est passée la semaine ? ». Identifier les causes : palmarès des thèmes récurrents, séparément pour le positif et le négatif. Un pourcentage dit qu'il y a un problème ; il ne dit pas lequel.

Le projet n'est pas un classifieur de sentiment (moyen, pas fin), pas un outil de réponse au client (pilotage, pas relation individuelle), pas un entraînement de modèle. Le ML est consommé comme un service ; le travail original est l'agrégation périodique.

3. Réponse apportée : deux chaînes distinctes

Chaîne entièrement serverless sur AWS, organisée en deux traitements de nature différente. Les confondre est l'erreur classique sur ce type de sujet.

Temps réel — analyser. Chaque fichier déposé sous `incoming/` déclenche la Lambda `traiter-avis`. Deux appels Comprehend (`detect_sentiment`, `detect_key_phrases`), puis écriture dans DynamoDB Avis. Résultat unitaire : un état intermédiaire, presque sans valeur métier pris isolément.

Périodique — synthétiser. Chaque lundi à 08h00 (Europe/Paris), `generer-rapport-hebdo` relit les sept jours écoulés et produit proportions, tendance, volume quotidien, comparaison à la semaine précédente, et surtout le palmarès des thèmes, illustré d'extraits d'avis. Le rapport est persisté puis envoyé par e-mail (SES).

> Pourquoi séparer. Le rejeu de l'agrégation est quasi gratuit (lectures DynamoDB) ; celui de l'analyse ne l'est pas (Comprehend facturé à l'appel). Les thèmes sont donc détectés à l'agrégation, pas à l'ingestion. On change une expression régulière, on relance la Lambda hebdomadaire, et tout l'historique est réinterprété sans un seul nouvel appel ML.

	Chaîne temps réel	Chaîne périodique
Déclencheur	S3 ObjectCreated sur incoming/*.txt	EventBridge Scheduler, lundi 08h00
Fréquence	À chaque avis	Une fois par semaine
Coût dominant	Appels Comprehend	Lectures DynamoDB (négligeables)
Rejouable ?	Non (nouvel appel facturé)	Oui, gratuitement
Livrable	Avis analysé (état intermédiaire)	Synthèse actionnable

4. Architecture détaillée

Région unique eu-west-1 (Irlande) : Comprehend n'est pas disponible à Paris. Runtime Python 3.12 sur arm64 (Graviton). Aucune dépendance tierce : boto3 est fourni par Lambda ; la logique métier vit dans la couche partagée `nordichome`, testable hors ligne. IAM au plus juste : la Lambda d'ingestion n'a qu'un droit d'écriture sur Avis.

Étape	Service	Rôle
Dépôt	S3 (incoming/)	Réception des avis bruts .txt ; cycle de vie 90 jours
Analyse	Lambda traiter-avis	Orchestration S3 → Comprehend → DynamoDB
ML	Amazon Comprehend	Sentiment + phrases clés, français natif
Stockage unitaire	DynamoDB Avis	PK avis_id ; GSI avis-par-mois (mois, date)
File d'échec	SQS DLQ	Uniquement après épuisement des réessais Lambda
Planification	EventBridge Scheduler	cron(0 8 ? * MON *), fuseau Paris
Agrégation	Lambda generer-rapport-hebdo	Stats, thèmes, comparaison N-1
Stockage synthèse	DynamoDB Rapports	PK semaine_id (ISO) ; GSI par date
Notification	SES	E-mail HTML + texte ; échec SES ≠ échec rapport
API	HTTP API + JWT Cognito	GET /rapports, GET /avis ; anonyme = 0 invocation
Interface	S3 site statique	Dashboard ; les données restent derrière l'API
Supervision	CloudWatch + SNS	3 alarmes e-mail vers le manager

4.1 Chaîne temps réel

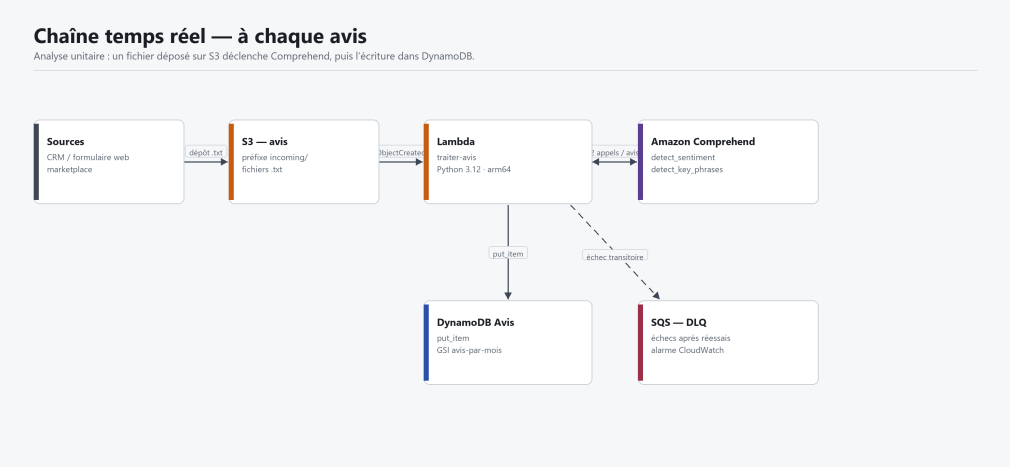


Figure 1 — Chaîne temps réel. Le chemin SQS est un chemin d'échec, pas le flux métier.

Convention de fichier : `AV001_2026-08-17_CLI-1042.txt` (avis_id, date, client_id). Le corps est du texte brut : n'importe quel système amont peut alimenter le pipeline. Le filtre S3 (préfixe + suffixe) est posé côté événement : une Lambda non invoquée coûte zéro.

Erreurs à deux vitesses. Définitive (nom invalide, fichier vide) : journalisée et ignorée — réessayer donnerait le même résultat. Transitoire (throttling Comprehend, indisponibilité DynamoDB) : l'exception remonte, Lambda réessaie (invocation asynchrone S3, 2 retries), puis l'événement bascule en DLQ. CloudWatch alarme si ≥ 1 erreur Lambda ou ≥ 1 message visible en file, sur 5 minutes ; SNS notifie le manager. `TreatMissingData: notBreaching` : l'absence d'invocation n'est pas un incident.

L'écriture DynamoDB est un `PutItem` idempotent sur `avis_id` : re-déposer le même fichier écrase l'analyse précédente au lieu de créer un doublon. La Lambda ne lit pas la table Avis.

4.2 Chaîne périodique

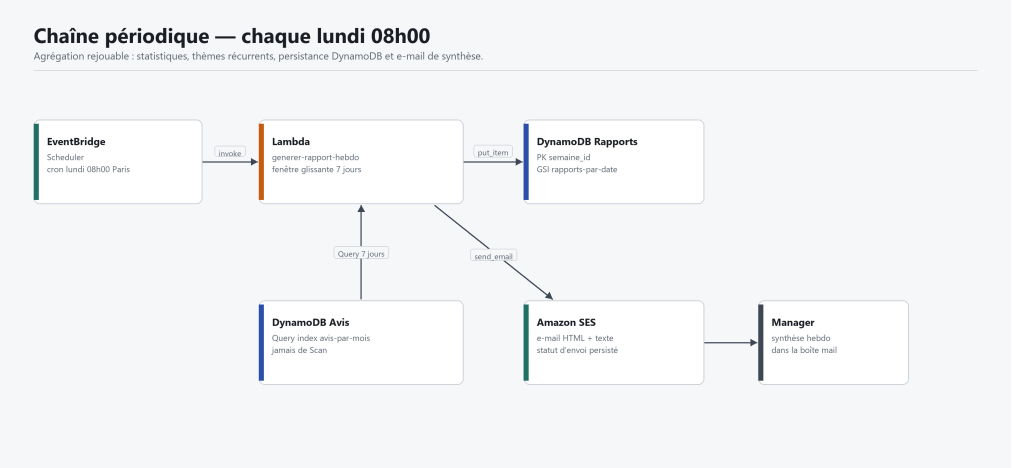


Figure 2 — Chaîne périodique. Query sur l'index mensuel, jamais de Scan de la table.

EventBridge Scheduler (pas une règle classique) pour gérer le fuseau Paris, y compris l'heure d'été. Charge utile identique à une invocation manuelle de démo : `{« jours »: 7, « envoyer_email »: true}`. Trois étapes : (1) collecte des 7 jours s'arrêtant la veille, via le GSI `avis-par-mois` (une semaine traverse au plus deux mois) ; (2) statistiques et tendance (`positive` / `negative` / `mitigee` si l'écart dépasse 15 points) ; (3) projection sur six thèmes stables, puis persistance + e-mail.

Modèle de données. Table Avis : partition mensuelle pour éviter la partition chaude d'une clé constante (plafond 1 000 WCU/s) tout en bornant le Query. Table Rapports : PK `semaine_id` (ex. 2026-W34) ; GSI `rapports-par-date` pour le listing du dashboard, 52 écritures par an. Facturation à la demande : le flux d'avis est trop irrégulier pour du provisionné.

4.3 Restitution et observabilité

HTTP API (~70 % moins chère qu'une REST API) avec autoriseur JWT natif Cognito : une requête non authentifiée n'atteint jamais la Lambda. Le front présente l'ID token (seul à porter `aud`). Pool `AllowAdminCreateUserOnly` : pas d'auto-inscription. Le bucket dashboard est public (code de la page) ; les données passent exclusivement par l'API. En production : CloudFront + OAC, bucket privé, HTTPS.

5. Des mots aux thèmes — le cœur métier

Comprehend extrait « le colis », « la porte cassée », « une charnière arrachée » : trois formulations d'un même problème. Un classement brut serait illisible et incomparable d'une semaine à l'autre. Le traitement se fait en deux temps : normalisation française (`nordichome.texte` : minuscules, accents, apostrophe, déterminants) puis projection sur un lexique de six thèmes (`nordichome.themes`).

Thèmes à polarité attendue négative	Thèmes à polarité attendue positive
Retards de livraison	Accueil en boutique
Colis et produits endommagés	Réactivité du service client
SAV difficile à joindre	Qualité des produits

La polarité « attendue » ne sert qu'à l'affichage. Le rangement dans le palmarès suit exclusivement le sentiment Comprehend de l'avis. Un avis négatif sur la qualité produit fait remonter ce thème côté négatif : c'est le signal qu'une force se dégrade. NEUTRAL et MIXED comptent dans le volume, pas dans les palmarès. Comportement verrouillé par test.

6. Jeu de données et validation

28 avis synthétiques en français, lundi 17 au dimanche 23 août 2026 (semaine ISO 2026-W34), quatre par jour. Polarité annotée : 13 négatifs (46 %), 11 positifs (39 %), 3 neutres, 1 mitigé — semaine dégradée mais non catastrophique, pour forcer la lecture des thèmes. Le champ `themes_attendus` n'est jamais envoyé à AWS.

Simulation hors ligne : 31 thèmes injectés, 31 retrouvés (rappel 100 %), 2 détections supplémentaires sur un avis neutre (sans effet sur le palmarès). 41 tests automatisés (normalisation, lexique, fenêtre à cheval sur deux mois, tendance, corpus vide, nommage, troncature UTF-8 Comprehend).

7. Pertinence, coût et alternatives

Le serverless colle au profil de charge : quelques dizaines d'avis par semaine, une exécution lundi matin, une poignée de consultations. Un serveur, même petit, serait facturé 168 heures pour quelques secondes utiles. Ici, l'absence d'avis coûte zéro. Le ML managé lève la barrière « sans expertise data ». Passer de 30 à 3 000 avis/semaine ne demande aucun changement d'architecture.

Service	Volume / mois (500 avis)	Free tier	Coût estimé
Comprehend	1 000 unités (2 appels × 500)	50 000 / mois × 12 mois	0 € puis ~0,25 €
Lambda	~600 invocations	1 M / mois, à vie	0 €
DynamoDB on-demand	~1 500 W + 3 000 R	25 Go stockage	< 0,01 €
S3, HTTP API, Cognito, SES, Scheduler, Logs	usage de démo	largement couvert	0 €

Total : nul pendant le projet, quelques dizaines de centimes ensuite. Le poste dominant à grande échelle est Comprehend — d'où l'intérêt de ne jamais rejouer l'analyse unitaire.

Alternatives écartées. BERT / CamemBERT : corpus et infra d'inférence, contredit « sans expertise data ». LDA / clustering : groupes instables, non nommés, incomparables d'une semaine à l'autre. Topic Modeling Comprehend : conçu pour de gros lots. QuickSight : facturé par utilisateur. ECS Fargate : facturé à la durée, sans intérêt pour quelques secondes par semaine.

8. Périmètre, organisation du code et limites

Dans le périmètre	Hors périmètre
Ingestion d'avis texte, analyse de sentiment, thèmes hebdo, e-mail, dashboard manager	Réponse automatique au client, modération, scoring individuel CRM
Authentification managers (Cognito, comptes créés par un admin)	Auto-inscription, multi-tenant, multi-langue
Supervision (logs 14 jours, 3 alarmes SNS)	Détection d'anomalie temps réel, CloudFront / certificat (écarté pour le bac à sable)

Couche partagée. `nordichome.texte`, `.themes`, `.fichiers` et `.agregation` sont montés en Lambda Layer (`/opt/python`). L'agrégation ne connaît ni boto3 ni les variables d'environnement : `scripts/simulation_locale.py` produit le rapport complet sans aucune ressource AWS. Le nom du bucket avis est calculé (`!Sub`) pour rompre le cycle CloudFormation Bucket → Fonction → Rôle → Bucket.

Chemin	Rôle
template.yaml	Infrastructure complète (AWS SAM)
src/traiter_avis/	Lambda #1 — S3 → Comprehend → DynamoDB
src/generer_rapport_hebdo/	Lambda #2 — synthèse + SES
src/api_rapports/	Lambda #3 — GET /rapports, GET /avis
frontend/	Dashboard statique (pas d'étape de build)
tests/	41 tests de la logique métier

Limites assumées. Lexique écrit à la main, sans négation (« pas de retard » active le motif retard) — le filtre par sentiment limite les dégâts. 28 avis démontrent le mécanisme, pas une performance statistique. Une seule langue (`fr`). SES en bac à sable. Dashboard en HTTP. Couplage fort à AWS, assumé pour une PME. Comprehend reste une boîte noire. Pistes : `detect_syntax` pour les portées de négation, Topic Modeling au-delà de quelques milliers d'avis, CloudFront + OAC en production.

9. Conclusion

Les trois exigences sont tenues : automatisation, agrégation périodique, thèmes dans les deux polarités. Sur le corpus de démo, 28 avis produisent sans intervention un rapport à tendance mitigée (46 % de négatifs), dominé par les retards, tandis que qualité produit et accueil boutique ressortent du côté positif. Ce qui fait la solution n'est pas la sophistication d'une brique, c'est la séparation analyse unitaire / synthèse périodique, qui rend le rejeu métier gratuit. Pour une PME à quelques centaines d'avis par mois, sans équipe technique ni budget d'outillage, le rapport valeur / coût d'exploitation fonde la pertinence de l'approche.